This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

FedAWA: Adaptive Optimization of Aggregation Weights in Federated Learning Using Client Vectors

Changlong Shi[1], He Zhao[2], Bingjie Zhang[1], Mingyuan Zhou[3], Dandan Guo[1*], Yi Chang[1*]
Jilin University, China[1] CSIRO's Data61, Australia[2]
The University of Texas at Austin, USA[3]
{shicl22,zhangbj24}@mails.jlu.edu.cn,he.zhao@data61.csiro.au
mingyuan.zhou@mccombs.utexas.edu, {guodandan,yichang}@jlu.edu.cn

Abstract

Federated Learning (FL) has emerged as a promising framework for distributed machine learning, enabling collaborative model training without sharing local data, thereby preserving privacy and enhancing security. However, data heterogeneity resulting from differences across user behaviors, preferences, and device characteristics poses a significant challenge for federated learning. Most previous works overlook the adjustment of aggregation weights, relying solely on dataset size for weight assignment, which often leads to unstable convergence and reduced model performance. Recently, several studies have sought to refine aggregation strategies by incorporating dataset characteristics and model alignment. However, adaptively adjusting aggregation weights while ensuring data security—without requiring additional proxy data—remains a significant challenge. In this work, we propose Federated learning with Adaptive Weight Aggregation (FedAWA), a novel method that adaptively adjusts aggregation weights based on client vectors during the learning process. The client vector captures the direction of model updates, reflecting local data variations, and is used to optimize the aggregation weight without requiring additional datasets or violating privacy. By assigning higher aggregation weights to local models whose updates align closely with the global optimization direction, FedAWA enhances the stability and generalization of the global model. Extensive experiments under diverse scenarios demonstrate the superiority of our method, providing a promising solution to the challenges of data heterogeneity in federated learning.

1. Introduction

Federated learning (FL) as an innovative paradigm in machine learning, has garnered significant attention in recent years [15, 31, 46]. This distributed optimization method leverages multiple local clients to collaboratively train a shared global model without sharing the client data, thereby preserving privacy and enhancing security. FL has a wide range of applications in many areas, such as healthcare [34], finance [4], and education [9]. The FedAvg [30] algorithm, a cornerstone method in FL, stores a global model on a central server. During the training process, this global model is distributed to participating clients for local updates. After each client updates its model, the server collects and aggregates the optimized parameters from the clients to update the global model. In FedAvg, the aggregation weights are determined based on the size of the local datasets. However, the performance of the model optimized by FedAvg tends to degrade due to data heterogeneity, which arises from user behaviors, preferences, devices, organizations, and other diverse factors [21]. As a result, different local models tend to optimize towards distinct local objectives, causing divergent optimization directions and unstable convergence [22], and ultimately degrading the overall model performance. This phenomenon has been both theoretically and empirically validated in [23, 39].

To mitigate the negative effects of data heterogeneity, several approaches have been proposed that adjust the aggregation weights on the server side to reduce bias during the model aggregation process. FedDisco [43] leverages both dataset size and the discrepancy between local and global category distributions to determine more distinguishing aggregation weights. Similar to FedAvg, FedDisco also uses fixed aggregation weights throughout the training process, which limits its ability to adapt to the dynamic optimization process. L-DAWA [33] employs cosine similarity between local models and the global model as aggregation weights, preventing deviations in the aggregation process from the global optimization direction. However, the similarity between models may not fully capture the relationship between clients, and directly using it as aggregation weights may lack sufficient adaptability. FedLAW [24] identifies the global

*Corresponding authors.

weight shrinking phenomenon and learns the optimal aggregation weights at the server with a proxy dataset, which is assumed to have the same distribution as the global dataset. This raises privacy concerns, which are of paramount importance in federated learning. Therefore, adaptively adjusting aggregation weights while ensuring data security remains a significant challenge.

To this end, we draw inspiration from recent work on task arithmetic [14, 41]. It suggests that changes in model parameters during training (referred to as task vectors) often capture meaningful information about the datasets and can be directly manipulated through arithmetic operations. Since direct access to clients' local data is not feasible in the federated learning context, we hope to leverage a similar approach about task arithmetic, $i.e.$, the changes in model parameters after local training, to infer information about the clients' local data. Given that the heterogeneity in federated learning primarily stems from differences in clients' local data, in this paper, we replace the original concept of the *task vector* with *client vector*, which is derived by subtracting the global model parameters from the locally trained client model parameters. We then conduct experiments to investigate the relationship between the client vector and local data, empirically demonstrating that client vectors capture the variations among local datasets. Furthermore, the aggregated client vectors align more closely with the desired direction of model updates. These findings could serve as a valuable guide for improving the model aggregation process in federated learning.

Building on these observations, we introduce Federated learning with Adaptive Weight Aggregation (FedAWA), a method that requires no additional datasets and enables dynamic adjustment of aggregation weights throughout the training process. FedAWA optimizes the aggregation weights on the server side, assigning higher weights to local models whose client vectors are more aligned with the overall update direction, thereby enhancing both the global model generalization and the training stability. To validate the effectiveness of FedAWA, we conduct extensive experiments across diverse scenarios. The contributions of this work are summarized as follows:

- We investigate the relationship between model merging and federated learning, designing the client vector to optimize aggregation weights in federated learning.
- We introduce FedAWA, a simple yet effective method for adaptively adjusting aggregation weights on the server side. FedAWA requires no additional data or transmission of original data, thus raising no privacy concern.
- We conduct extensive experiments across diverse scenarios, demonstrating the effectiveness of FedAWA.

2. Related Works

Federated Learning. Federated learning is a rapidly advancing research field with many remaining open problems to address. One of the primary issues that can significantly degrade its performance is data distribution heterogeneity. Research addressing this problem can generally be categorized into two main directions: client-side and server-side adjustment. First is the client-side drift adjustment. Due to the data heterogeneity, local models trained on the clients may exhibit different degrees of bias, thereby affecting the performance of the global model. Many methods aim to reduce this bias by adjusting the training process of local models [8, 19, 25]. FedProx [22] utilizes the l_2-distance between the global model and the local model as a regularization term during the training of the local model. FedDyn [1] proposes a dynamic regularizer for each client to align the global and local solutions. These methods merely assign aggregation weights based on the size of the local dataset. In contrast, our approach focuses on the server-side aggregation process, making it easily combined with these methods to enhance model performance further. Several other studies focus on adjusting the model on the server side [5, 26]. FedDisco [43] leverages both dataset size and the discrepancy between local and global category distributions to design the fixed aggregation weights. FedLAW [24] revisits the weighted aggregation process and utilizes a proxy dataset, which is assumed to have the same distribution as the global dataset, to learn the optimal global weight shrinking factor and the aggregation weights. In contrast, our proposed FedAWA eliminates the need for proxy datasets, while also enables the dynamic optimization of aggregation weights. L-DAWA [33] employs cosine similarity between local models and the global model as aggregation weights. Both L-DAWA and ours can adaptively adjust the aggregation weights during the training process. The key difference is that we optimize the aggregation weights rather than directly using the similarity as the aggregation weight. Besides, we leverage the variations of local model and global model before and after training to design the client vector and global vector, which better reflects the local data information.

Model Merging. Recently, with the rapid advancements in deep learning, model merging has garnered significant attention [2, 29, 40]. This technique aims to aggregate multiple well-trained models into a single unified model, thereby inheriting their individual capabilities without incurring the computational overhead and complexity associated with traditional ensembling methods. Task Arithmetic [14] stands out as a simple yet highly effective method for model merging. It introduces task vectors, which are both efficient and lightweight, facilitating improved generalization across tasks. Ties-Merging [41] identifies redundancy and sign conflicts in direct task vector aggregation and proposes three steps to resolve them. AdaMerging [42] addresses the limitation

of shared merging coefficients in prior methods, designing separate aggregation weights for each task vector to enhance model adaptability. The ability of model merging to aggregate models without relying on training data closely aligns with the requirements of federated learning, where preserving data privacy is of paramount importance. However, model merging is a single-step aggregation and the federated learning needs a multiple communication rounds, where the former already owns well-trained local models and the latter requires constant iterative updating of the global and local models. The key difference makes it difficult to design the aggregation weights in federated learning using the model merging directly, which is ours research focus in this work. To the best of our knowledge, we are the first to explore the relationship between model merging and federated learning, and introduce an adaptive optimization of aggregation weights by designing the client vector in federated learning.

3. Background

Federated Learning. Federated Learning consists of K clients and a central server, where each client has its own private local dataset $\mathcal{D}_k$. FL aims to enable clients to collaboratively learn a global model for the server without data sharing. In communication round t (out of a total of T rounds), the parameters of the global model and the client k's model are denoted as θ_g^t and θ_k^t, respectively. The workflow of the basic FL method, FedAvg [30], in communication round t can be described as follows:

- **Step 1:** Server broadcasts the parameters of global model θ_g^t to each client;
- **Step 2:** Each client k performs E epochs of local model training on private dataset $\mathcal{D}_k$ to obtain a local model θ_k^t;
- **Step 3:** Clients upload the local models to the server;
- **Step 4:** Server merges the local models to get a new global model: $\theta_g^{t+1} = \sum_{k=1}^{K} \lambda_k \theta_k^t$, where λ_k is the aggregation weight of the client k and FedAvg sets $\lambda_k = \frac{|\mathcal{D}_k|}{\sum_{i=1}^{K} |\mathcal{D}_i|}$.

However, simply using dataset size as aggregation weights in step 4 is suboptimal when local data exhibits high heterogeneity [20, 23, 39]. Various methods have attempted to address this issue by adjusting the aggregation scheme [24, 33, 43], but they still face challenges such as lack of adaptability and the requirement for the proxy dataset. In this paper, we aim to explore a method that can adaptively adjust the model aggregation weights ($\boldsymbol{\lambda}$ in step 4) during the training process, while eliminating the need for a proxy dataset. This method is designed to enhance both the performance of the global model and privacy security.

Task Arithmetic. Task Arithmetic [14] stands out as a straightforward yet highly efficient technique for merging models. It introduces task vector, which is defined as the difference between the fine-tuned and pre-trained model parameters, i.e., $\tau_k = \theta_k^* - \theta_0$, where θ_0 represents the pre-trained model parameters, and θ_k^* refers to the model parameters fine-tuned on the downstream task k. The core idea of this approach involves summing task vectors, which are scaled and added to the pre-trained model parameters to compute the final parameters of the merged model. This can be mathematically formulated as $\theta' = \theta_0 + \lambda \sum_{i=1}^{n} \tau_i$, where λ is the scaling coefficient, and θ' represents the aggregated model parameters. Task arithmetic's advantage lies in its ability to integrate model parameters without needing access to the original training data, while producing a merged model that performs well across tasks. This capability closely aligns with the requirements of federated learning, where preserving data privacy is of paramount importance. However, task arithmetic is typically characterized by a single-step model aggregation, and federated learning involves multiple communication rounds with iterative local model training on the client side and model aggregation on the server side, which is the major challenge in federated learning.

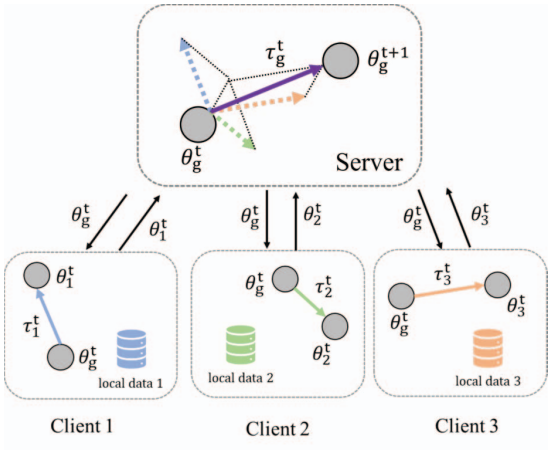


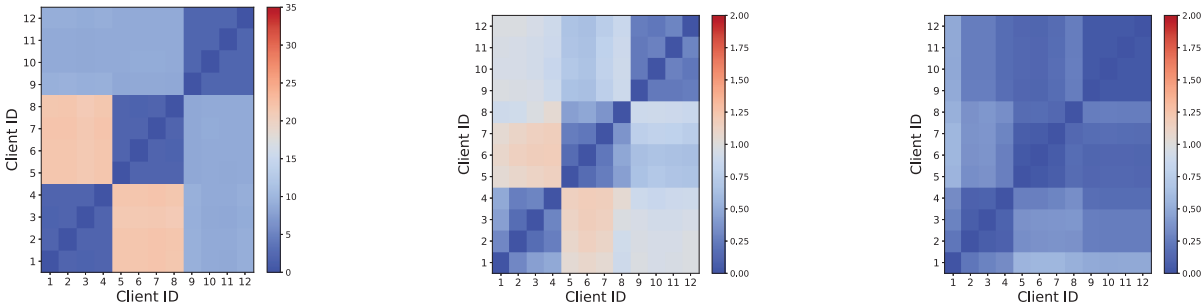
Figure 1. The illustration of client vectors. τ_k^t is the client vector of the k-th client, and τ_g^t represents the global vector obtained by aggregating the client vectors.

4. Method

In this section, we introduce Federated Learning with Adaptive Weight Aggregation (FedAWA), a method that adaptively optimizes the aggregation weights without relying on a proxy dataset, thereby enhancing model performance while effectively addressing privacy concerns. Below, we introduce the details of our proposal.

4.1. Motivation

The motivation for our work derives from the task arithmetic in the field of model merging that can reflect the task-related information without data leakage. In the context of federated learning, the server distributes the global model to clients for training at each communication round. The global model serves as the "per-trained" model, while training different



(a) Difference between client datasets. (b) Difference between client vectors. (c) Difference between client models.

Figure 2. Differences between client local datasets $\mathcal{D}_{1:K}$, client vectors $\tau^t_{1:K}$, and client models $\theta^t_{1:K}$.

client models based on the respective heterogeneous local data can be considered as different "tasks". Given that federated learning heterogeneity mainly arises from differences in client local data, we replace the *task vector* with the *client vector*, defined as the difference between the global model parameters and the locally trained client model parameters. A more intuitive illustration is shown in Figure 1. In the t-th communication round, the client vector of the k-th client τ^t_k and the merged global vector τ^t_g are defined as follows:

$$\tau^t_k = \theta^t_k - \theta^t_g, \quad \tau^t_g = \sum_{k=1}^{K} \lambda^t_k \tau^t_k, \tag{1}$$

where λ^t_k is the learnable aggregation weight at the t-th communication round. Building on previous research [14, 42], we hypothesize that the client vector can more efficiently encapsulate relevant information about the local data. This information can then be leveraged to optimize the aggregation weights in federated learning, all while preserving data privacy. Then, we empirically validate this hypothesis through experiments.

4.2. Empirical Observations

Client Vector and Local Data. We first investigate the relationship between the client vectors and the local data to demonstrate that the client vectors effectively capture the variations between different local data and reflect the corresponding differences among the local datasets. In Figure 2b, we illustrate the divergence between different local datasets and between the corresponding client vectors. As shown, the difference between client vectors in Figure 2b closely resemble those of the local dataset in Figure 2a. The distance between models parameters, as shown in Figure 2c, fails to effectively reflect the relationships between local datasets. This demonstrates that , compared to previous methods [33] relying on overall model parameters, the client vector provides a more accurate representation of the information in local datasets. Hence, we explore the possibility of leveraging this phenomenon to enhance the model aggregation process in federated learning. More implementation details can be found in the Appendix A.1.

Ideal Vector. Here, we explore the relationship between the merged global vector and the ideal vector, where the former is computed with the size of the local datasets being the aggregation weight in (1) and the latter is explained below. Ideally, federated learning aims to aggregate local client models into a global model that matches the performance of a model trained directly on the global dataset $\mathcal{D}_g = \mathcal{D}_1 \cup \mathcal{D}_2 \cup ... \cup \mathcal{D}_K$, where $\mathcal{D}_k$ is the local dataset of the k-th client. Now, we view θ^t_g in (1) as the initialized global parameter and use the global dataset $\mathcal{D}_g$ to train the model, producing the parameter θ^t_{ideal}. Notably, in practical federated learning, clients do not share data with each other, and we only utilize the global dataset $\mathcal{D}_g$ to explore potential methods for enhancing the effectiveness of federated learning. Now, we can denote τ^t_{ideal} as the ideal vector in the t-th communication round, expressed as $\tau^t_{ideal} = \theta^t_{ideal} - \theta^t_g$. To explore whether aggregating the client vectors could similarly capture the update direction of the model trained on the global dataset, we compare the distance between merged global vector (and all client vectors) and the ideal vector τ^t_{ideal}. As shown in Figure 3, it can be observed that the aggregation of the client vectors τ^t_g is closer to the ideal vector τ^t_{ideal} compared to individual client vectors τ^t_k. This indicates that aggregating the client vectors can yield a pseudo global vector that reflects the overall data distribution across all clients. Therefore, we will utilize the client vector and the merged global vector as the guide to optimize the aggregation weights in federated learning.

4.3. FedAWA

Based on the aforementioned empirical observations, we suggest that in FL, the aggregation of client vectors can serve as an indicator to adjust the aggregation weights. Specifically, clients whose update directions closely align with the merged global vector should be assigned greater weights, reflecting their contributions that are more consistent with the overall learning objective. Conversely, clients with less

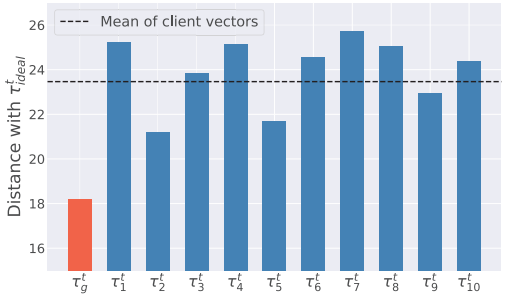


Figure 3. Distance with the ideal update vector τ^t_{ideal}.

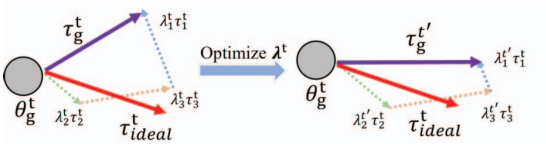


Figure 4. Adjustment of the aggregation weights.

Algorithm 1 FedAWA: Federated learning with Adaptive Weight Aggregation.

Input: Communication round T, local epoch E, local datasets $\{\mathcal{D}_1, ..., \mathcal{D}_K\}$, initial global model θ^1_g, initial aggregation weights $\boldsymbol{\lambda}^0$, where $\lambda^0_k = \frac{|\mathcal{D}_k|}{\sum_{i=1}^{K}|\mathcal{D}_i|}$;
Output: Final global model θ^T_g;
for $t = 1$ **to** T **do**
  Server sends global model θ^t_g to each client;
  # Clients execute:
  for each client $k \in [K]$ **do**
    $\theta^t_k \leftarrow$ ClientUpdate$(\theta^t_g, \mathcal{D}_k, E)$;
    Send θ^t_k to server;
  end for
  # Server executes:
  Server computes τ^t_k, τ^t_g through Equation 1.
  If FedAWA then: Server optimizes the aggregation weights $\boldsymbol{\lambda}^t$ according to Equation 3.
  If FedAWA-L then: Server optimizes the aggregation weights $\boldsymbol{\lambda}^t_l$ according to Equation 4.
  Server aggregates the local models to generate the global model:
  If FedAWA then: $\theta^{t+1}_g = \sum_{k=1}^{K} \lambda^t_k \theta^t_k$;
  If FedAWA-L then: $\theta^{t+1}_{gl} = \sum_{k=1}^{K} \lambda^t_{kl} \theta^t_{kl}$;
end for

aligned update directions can be assigned lower weights to minimize potential negative impacts on model convergence. An intuitive example is shown in Figure 4. By adjusting the aggregation weights, the deviation between the aggregated global vector and the ideal vector can be minimized, effectively reducing the variation among clients. Thus, we propose FedAWA by leveraging the relationship between client vectors and merged global vector to dynamically adjust the model aggregation weights, thereby enhancing the overall effectiveness and robustness of the aggregation process in federated learning. More specifically, the initial three steps of our method align with those outlined in Section 3. The server broadcasts the global model θ^t_g to each client, and the client sends the locally trained model θ^t_k back to the server. Then, we calculate the client vector of communication round t as $\tau^t_k = \theta^t_k - \theta^t_g$. We aim to optimize the aggregation weights by assigning higher weights to clients whose update directions are more alignment to the merged model vector τ^t_g. Subsequently, the distance between the client vectors and the derived merged model vector τ^t_g in (1) be used as a supervisory signal to optimize the aggregation weights, and the objective function can be expressed as:

$$\boldsymbol{\lambda}^t = \arg\min_{\boldsymbol{\lambda}} \left(\sum_{k=1}^{K} \lambda_k \|\tau^t_k - \tau^t_g\|_2 \right), \text{s.t.} \|\boldsymbol{\lambda}\|_1 = 1. \quad (2)$$

Through the function, clients whose update directions more align with the merged global vector are assigned higher aggregation weights, while those with less aligned directions receive lower weights. This approach optimizes the global model's update trajectory, ultimately enhancing overall model performance. In addition, we also incorporate a global alignment component to ensure that the aggregated global model does not deviate excessively from the previous communication round's global model, thus maintaining the stability of the training process. To achieve this, we introduce an additional regularization term and rewrite the objective function (2) as follows:

$$\boldsymbol{\lambda}^t = \arg\min_{\boldsymbol{\lambda}} \left(\sum_{k=1}^{K} \lambda_k \|\tau^t_k - \tau^t_g\|_2 + \mathrm{d}(\sum_{k=1}^{K} \lambda_k \theta^t_k, \theta^t_g) \right), \quad \text{s.t.} \|\boldsymbol{\lambda}\|_1 = 1, \quad (3)$$

where $\mathrm{d}(\cdot,\cdot)$ represents the distance function. In this paper, we implement it using 1 - cosine similarity. In the experiments, we also evaluate the impact of different distance metrics on model performance. Following the optimization procedure outlined above, the aggregation weight $\boldsymbol{\lambda}^t$ is derived and subsequently utilized in step 4 in Section 3 to produce the final global model $\theta^{t+1}_g = \sum_{k=1}^{K} \lambda^t_k \theta^t_k$.

Previous studies have demonstrated significant divergence across various layers of deep neural networks [17, 28, 33]. Given that each layer in deep neural networks may vary differently, it might be beneficial to design specific aggregation weights for each layer of the model. Consequently, we extend our method by calculating separate aggregation weights for each layer and propose FedAWA-L, which allows for more fine-grained adjustments in the model aggregation process. The objective function of the layer-wise method as:

Table 1. Top-1 test accuracy (%) on CIFAR-10, CIFAR-100, and TinyImageNet datasets with $\alpha = 0.5$, $\alpha = 0.1$, and $\alpha = 100$.

Dataset	CIFAR-10			CIFAR-100			TinyImageNet			**Average**
Heterogeneity	NIID(α=0.1)	NIID(α=0.5)	IID(α=100)	NIID(α=0.1)	NIID(α=0.5)	IID(α=100)	NIID(α=0.1)	NIID(α=0.5)	IID(α=100)	
FedLAW [24]	64.76	75.27	81.30	34.59	37.56	41.05	29.13	33.49	37.20	48.26
FedAvg [30]	61.04	74.47	76.01	36.71	41.08	41.46	29.44	34.43	36.31	47.88
FedDisco [43]	62.86	74.72	75.40	36.46	41.02	41.46	31.19	34.29	36.31	48.19
L-DAWA [33]	62.87	75.61	76.10	36.31	39.81	42.39	32.02	31.43	36.25	48.09
FedProx [22]	60.62	73.27	73.96	34.60	39.35	38.15	29.37	34.32	35.03	46.52
FedAdam [32]	61.76	73.04	70.40	32.12	34.92	30.37	21.77	27.39	24.08	41.76
FedDyn [1]	56.37	74.61	77.92	36.92	**44.80**	41.04	27.32	32.80	34.32	47.34
FedAWA (Ours)	63.55	75.65	**80.10**	**37.04**	41.89	42.84	33.07	34.57	**36.59**	49.48
FedAWA-L (Ours)	**65.13**	**75.99**	79.70	36.85	42.52	**45.27**	**33.42**	**34.86**	36.04	**49.98**

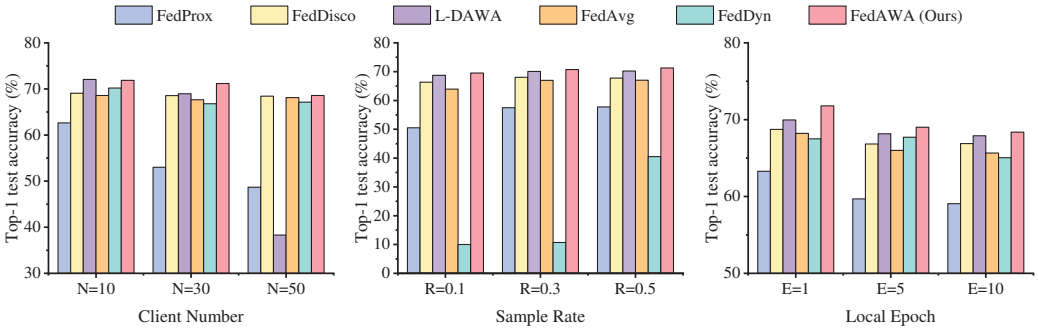


Figure 5. Top-1 test accuracy (%) on different client numbers, sample rates, and local epochs.

$$\boldsymbol{\lambda}_l^t = \arg\min_{\boldsymbol{\lambda}} \left(\sum_{k=1}^{K} \lambda_{kl} \|\tau_{kl}^t - \tau_{gl}^t\|_2 + \mathrm{d}(\sum_{k=1}^{K} \lambda_{kl} \theta_{kl}^t, \theta_{gl}^t) \right), \quad \text{s.t.} \|\boldsymbol{\lambda}\|_1 = 1, \tag{4}$$

where λ_{kl} represents the aggregation weight for the l-th layer of the k-th client model. τ_{kl}^t and θ_{kl}^t represent the l-th layer of the client vector and the local model for client k, respectively, while τ_{gl}^t and θ_{gl}^t denote the l-th layer of the global vector and the global model.

Through the aforementioned method, we can adaptively optimize the aggregation weights during the federated learning training process without requiring additional information. All the utilized information comes from the existing information within the basic federated learning algorithm, thereby ensuring privacy security. The pseudo-code of FedAWA is shown in Algorithm 1.

4.4. Discussions

Privacy. FedAWA offers enhanced privacy protection and practical adaptability over prior approaches [24, 43]. Instead of relying on fine-tuning with a proxy dataset, it directly optimizes aggregation weights by leveraging model parameters and gradient updates, both of which are easily accessible to the server during training. By eliminating the need for additional data, FedAWA mitigates the risk of data leakage, making it more applicable to real-world environments.

Modularity. Our proposed FedAWA can serve as a plug-and-play module for many existing FL methods, enhancing their performance across a wide range of applications. For FL methods that adjust the client-side model [1, 19, 22], FedAWA operates on the server-side, making it easily integrable with these client-side adjustments. Furthermore, for methods that adjust the server-side model [11, 30, 43], which use fixed aggregation weights, these weights can be used as initial values in our optimisation process, allowing for further optimisation and improved model performance.

5. Experiments

5.1. Experiment Setup

Dataset and Baselines. In this study, we consider three image classification datasets: CIFAR-10 [16], CIFAR-100 [16], and Tiny-ImageNet [6]. For each dataset, all methods are evaluated with the same model architectures for a fair comparison. In Table 1, We use ResNet20 [10] for CIFAR-10 and CIFAR-100, ResNet18 [10] for Tiny-ImageNet. We compare our method with seven representative baselines. Specifically, (1) FedAvg [30] serves as the standard algorithm for Federated Learning; (2) FedProx [22], and FedDyn [1] focus on local model adjustments; (3) FedAdam [32], FedDisco [43], FedLAW [24] and L-DAWA [33] operate on the server side. Where FedDisco and L-DAWA specifically emphasize the aggregation scheme adjustments, making them highly relevant to our proposed method. Additionally, we also show the performance of FedLAW [24]. However, since it leverages additional data for fine-tuning that other methods do not, we present it only for reference. More details about

the experimental setup can be found in Appendix A.[1] To emulate the federated learning scenario, we randomly partition the training dataset into K groups, assigning group k to client k. In practical FL scenarios, clients often exhibit heterogeneity, resulting in Non-IID characteristics in their data. To simulate this heterogeneity, we employ Dirichlet sampling, denoted as Dir_α, which is widely used in FL literature [38, 43, 44]. A smaller α value corresponds to greater Non-IID characteristics. For a fair comparison, we apply the same data synthesis approach across all methods.

5.2. Main Results

Performance. In this section, we compare our proposed FedAWA with all baselines and report the test accuracy on all datasets, as shown in Table 1. It can be observed that FedAWA achieves the overall best performance across various datasets and heterogeneity settings, highlighting the effectiveness of our proposed method. In addition, FedAWA-L achieves better performance compared to FedAWA, indicating that the layer-wise approach enables finer model aggregation, which provides a more detailed and precise optimization process, leading to improved results. However, this improvement comes at the cost of higher computational overhead, as aggregation weights must be optimized separately for each layer of the model. Therefore, FedAWA strikes a more balanced trade-off between performance and computational cost. To evaluate the robustness of our method in different federated learning scenarios, we tune three crucial parameters of FL: the number of clients $K \in \{10, 30, 50\}$, the number of local epoch $E \in \{1, 5, 10\}$, and partial participation ratio $R \in \{0.1, 0.3, 0.5\}$. We show the result in Figure 5. The experiments consistently reveal that our proposed method consistently brings performance improvement across different FL settings.

Modularity. Our proposed FedAWA can be integrated with various existing FL methods to further enhance their performance. As illustrated in Figure 6, we evaluated the performance of our approach in combination with the foundational FL algorithm FedAvg, the client-side adjustment method FedProx, and the aggregation weight adjustment method FedDisco. Specifically, when combined with FedDisco, FedAWA initializes the aggregation weights using FedDisco's approach and subsequently optimizes them further using our proposed method. The results demonstrate that FedAWA consistently improves the performance of the baseline methods across varying degrees of data heterogeneity, further validating the effectiveness of our approach.

Computation Efficiency. In Table 2, we show the aggregation execution time of our method FedAWA, FedAWA-L and the closely related work FedAvg [30], L-DAWA [33], and FedLAW [24]. The used model architecture is ResNet20

[1]The source code is available at https://github.com/ChanglongShi/FedAWA

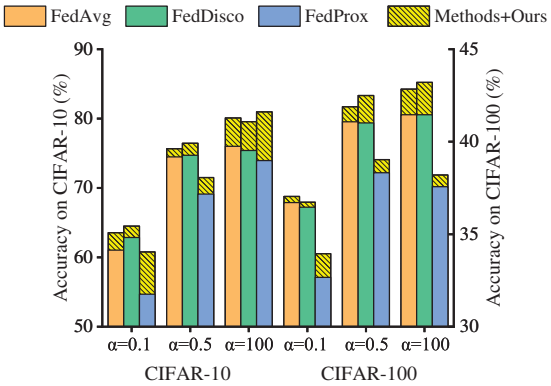


Figure 6. Modularity. Performance improvements achieved by integrating our proposed FedAWA with different FL algorithms under varying datasets and degrees of data heterogeneity.

Table 2. Average aggregation execution time. (ResNet20)

Method	FedAvg [30]	L-DAWA [33]	FedLAW [24]	**FedAWA**	**FedAWA-L**
Execution Time (Sec)	0.10	2.52	10.11	0.82	15.21

Table 3. The performance of compared methods with different model architectures.

Dataset	**Method**	CNN	ResNet20	WRN56_4	DenseNet121	ViT
CIFAR-10	FedLAW [24]	70.18	75.37	80.46	86.43	51.20
	FedAvg [30]	68.28	75.07	78.97	86.14	51.31
	FedDisco [43]	70.24	73.42	78.74	85.05	53.97
	L-DAWA[33]	70.87	75.79	81.51	86.29	53.43
	FedAWA (Ours)	**71.17**	**77.71**	**81.96**	**86.63**	**56.16**
CIFAR-100	FedLAW [24]	33.59	38.53	18.44	55.36	22.03
	FedAvg [30]	32.53	41.18	39.71	56.59	25.60
	FedDisco [43]	33.57	40.23	45.96	58.91	31.38
	L-DAWA [33]	32.55	41.23	48.22	61.45	30.71
	FedAWA (Ours)	**37.38**	**41.79**	**49.21**	**62.61**	**31.44**

and the dataset is CIFAR-10. For FedLAW, the proxy dataset contains 200 samples and the server epoch is 100 (consistent with [24]). It can be observed that, while FedAWA-L achieves the best performance, it incurs additional computational overhead compared to other methods. In comparison to L-DAWA and FedLAW, our proposed FedAWA significantly reduces execution time. While FedAWA incurs slightly higher computational costs than FedAvg, the balance between its computational requirements and the resulting performance remains acceptable.

5.3. Ablation Studies

Effects of Model Architectures. In Table 3, we evaluate our proposed FedAWA across a diverse range of model architectures, including CNN, ResNet [10], Wide-ResNet (WRN) [45], DenseNet [12], and Vision Transformer (ViT) [7]. The results highlight the effectiveness of FedAWA across these different architectures, demonstrating its robust performance not only as network depth and width increase but also when applied to models with distinct architectural designs.

Effects of Optimization. In this section, we conduct experiments to analyze the effectiveness of the optimization

Table 4. Comparison of directly using cosine similarity as the aggregation weights.

Dataset	CIFAR-10		CIFAR-100		**Average**
Heterogeneity	α=100	α=0.1	α=100	α=0.1	
FedAvg [30]	76.01	61.04	41.46	36.71	53.81
L-DAWA [33]	76.10	62.87	42.39	36.31	54.42
FedAWA-COS	78.89	62.14	42.31	36.74	55.02
FedAWA (Ours)	80.10	63.55	42.84	37.04	**55.88**

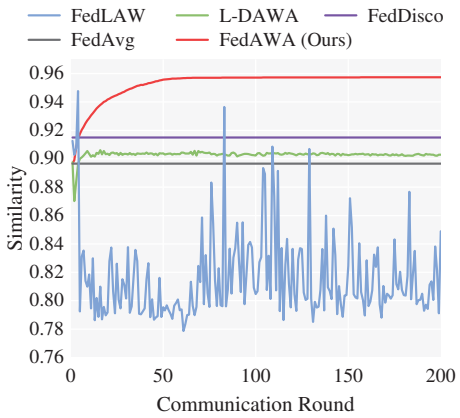


Figure 7. Similarity between aggregation weights and dataset vectors during training.

process. For comparison, we implement FedAWA-COS, where the aggregation weights are determined by directly computing the similarity between the client vectors and the global vector, rather than optimizing them using Equation (3). In Table 4, with FedAvg and L-DAWA as the baseline, we compare the performance of FedAWA-cos and FedAWA across different datasets and levels of data heterogeneity. The results in Table 4 demonstrate that FedAWA achieves higher model accuracy than FedAWA-COS, highlighting the benefits of iteratively optimizing the model aggregation weights to further improve model performance. Furthermore, compared to L-DAWA, which uses the model parameter similarity as the aggregation weight, FedAWA-COS achieves better performance, highlighting the client vector's ability to capture local data more effectively.

Aggregation Weights. In this section, we conduct experiments to observe the evolution of aggregation weights during the training process. We first calculate the similarity between the client's local dataset and the global dataset, forming a K-dimensional dataset vector. We treat this dataset vector as the ideal aggregation weights vector, where local datasets more similar to the global dataset receive higher weights, and vice versa. We then observe the similarity between the aggregation weights obtained by different methods and the dataset vector. A higher similarity with the dataset vector indicates better alignment with the ideal aggregation weights. The results are shown in Figure 7. As can be seen, our method steadily increases and eventually converges during training, with a higher similarity to the dataset vector compared to other methods, demonstrating the effectiveness of our FedAWA. FedAvg and FedDisco use fixed aggregation weights, so there is no change in similarity during training. L-DAWA exhibits a sharp initial decline followed by an increase, which can be attributed to its direct use of model similarity as aggregation weights, introducing greater randomness during the early stages of training. FedLAW displays considerable fluctuations, likely due to the influence of the shrinkage factor within its optimization process. More details can be found in Appendix A.2.

Table 5. The performance with varying distance metrics.

Dataset	**Method**	NIID($\alpha = 0.1$)	NIID($\alpha = 0.5$)
CIFAR-10	FedAWA-w/o reg	63.05	75.06
	FedAWA-w/ euc_reg	63.53	75.35
	FedAWA-w/ cos_reg	**63.55**	**75.65**
CIFAR-100	FedAWA-w/o reg	36.75	41.36
	FedAWA-w/ euc_reg	**37.20**	41.55
	FedAWA-w/ cos_reg	37.04	**41.89**

Effects of Regularization. We investigate the impact of the regularization term (the second term) in Equation 3 on model performance. The results are presented in Table 5, where we show the result for three different methods: without regularization term, using the Euclidean distance for the regularization term, and using 1 - cosine similarity for the regularization term, labeled as FedAWA-w/o reg, FedAWA-w/ euc_reg, and FedAWA-w/ cos_reg, respectively. As observed, omitting the regularization term leads to a decrease in performance, which demonstrates the effectiveness of the regularization. Furthermore, FedAWA-w/ cos_reg generally performs slightly better than FedAWA-w/ euc_reg. This might be because the 1-cosine similarity ranges from 0 to 2, providing more stability compared to the unbounded range of Euclidean distance.

6. Conclusion

In this paper, through empirical explorations, we demonstrate that client vectors in federated learning effectively capture relevant information about local datasets. Furthermore, we investigate the relationship between the ideal model update direction and the client vector. Building on these observations, we propose FedAWA, a method that adaptively optimizes aggregation weights without relying on a proxy dataset, thereby enhancing model performance while addressing privacy concerns. Experimental results show that FedAWA delivers outstanding performance across various settings. A potential limitation of our method lies in its applicability exclusively to scenarios where client model architectures are identical. Model heterogeneity among clients remains a prominent challenge in federated learning, and this limitation is shared by many existing FL methods. As part of future work, we aim to extend our approach to accommodate scenarios with heterogeneous client model architectures.

Acknowledgements

The work of Dandan Guo is supported by the National Natural Science Foundation of China (NSFC) under Grant 62306125. The work of Yi Chang is supported in part by the National Key R&D Program of China under Grant 2023YFF0905400, in part by the National Natural Science Foundation of China under Grant U2341229.

References

[1] Durmus Alp Emre Acar, Yue Zhao, Ramon Matas, Matthew Mattina, Paul Whatmough, and Venkatesh Saligrama. Federated learning based on dynamic regularization. In *International Conference on Learning Representations*, 2021. 2, 6

[2] Samuel Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *The Eleventh International Conference on Learning Representations*, 2023. 2

[3] David Alvarez-Melis and Nicolo Fusi. Geometric dataset distances via optimal transport. *Advances in Neural Information Processing Systems*, 33:21428–21439, 2020. 1

[4] David Byrd and Antigoni Polychroniadou. Differentially private secure multi-party computation for federated learning in financial applications. In *Proceedings of the First ACM International Conference on AI in Finance*, pages 1–9, 2020. 1

[5] Hong-You Chen and Wei-Lun Chao. Fedbe: Making bayesian model ensemble applicable to federated learning. In *International Conference on Learning Representations*, 2021. 2

[6] Patryk Chrabaszcz, Ilya Loshchilov, and Frank Hutter. A downsampled variant of imagenet as an alternative to the cifar datasets. *arXiv preprint arXiv:1707.08819*, 2017. 6, 1

[7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020. 7, 3

[8] Liang Gao, Huazhu Fu, Li Li, Yingwen Chen, Ming Xu, and Cheng-Zhong Xu. Feddc: Federated learning with non-iid data via local drift decoupling and correction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10112–10121, 2022. 2

[9] Song Guo, Deze Zeng, and Shifu Dong. Pedagogical data analysis via federated learning toward education 4.0. *American Journal of Education and Information Technology*, 10(2): 56–65, 2020. 1

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 6, 7, 3

[11] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Measuring the effects of non-identical data distribution for federated visual classification. *arXiv preprint arXiv:1909.06335*, 2019. 6

[12] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 7

[13] Wenke Huang, Mang Ye, Zekun Shi, He Li, and Bo Du. Rethinking federated learning with domain shift: A prototype view. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16312–16322. IEEE, 2023. 3

[14] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *The Eleventh International Conference on Learning Representations*, 2023. 2, 3, 4

[15] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning*, 14(1–2): 1–210, 2021. 1

[16] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 6, 1

[17] Sunwoo Lee, Tuo Zhang, and A Salman Avestimehr. Layer-wise adaptive model aggregation for scalable federated learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8491–8499, 2023. 5

[18] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. *Advances in neural information processing systems*, 31, 2018. 3

[19] Qinbin Li, Bingsheng He, and Dawn Song. Model-contrastive federated learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10713–10722, 2021. 2, 6

[20] Qinbin Li, Yiqun Diao, Quan Chen, and Bingsheng He. Federated learning on non-iid data silos: An experimental study. In *2022 IEEE 38th international conference on data engineering (ICDE)*, pages 965–978. IEEE, 2022. 3

[21] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *IEEE signal processing magazine*, 37(3):50–60, 2020. 1

[22] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems*, 2:429–450, 2020. 1, 2, 6

[23] Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. On the convergence of fedavg on non-iid data. In *International Conference on Learning Representations*, 2020. 1, 3

[24] Zexi Li, Tao Lin, Xinyi Shang, and Chao Wu. Revisiting weighted aggregation in federated learning with neural networks. In *Proceedings of the 40th International Conference on Machine Learning*. JMLR.org, 2023. 1, 2, 3, 6, 7

[25] Zexi Li, Xinyi Shang, Rui He, Tao Lin, and Chao Wu. No fear of classifier biases: Neural collapse inspired federated learning with synthetic and fixed classifier. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5319–5329, 2023. 2

[26] Tao Lin, Lingjing Kong, Sebastian U Stich, and Martin Jaggi. Ensemble distillation for robust model fusion in federated learning. *Advances in Neural Information Processing Systems*, 33:2351–2363, 2020. 2

[27] Mi Luo, Fei Chen, Dapeng Hu, Yifan Zhang, Jian Liang, and Jiashi Feng. No fear of heterogeneity: Classifier calibration for federated learning with non-iid data. *Advances in Neural Information Processing Systems*, 34:5972–5984, 2021. 1

[28] Xiaosong Ma, Jie Zhang, Song Guo, and Wenchao Xu. Layer-wised model aggregation for personalized federated learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10092–10101, 2022. 5

[29] Michael S Matena and Colin A Raffel. Merging models with fisher-weighted averaging. *Advances in Neural Information Processing Systems*, 35:17703–17716, 2022. 2

[30] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017. 1, 3, 6, 7, 8

[31] Pian Qi, Diletta Chiaro, Antonella Guzzo, Michele Ianni, Giancarlo Fortino, and Francesco Piccialli. Model aggregation techniques in federated learning: A comprehensive survey. *Future Generation Computer Systems*, 2023. 1

[32] Sashank J. Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and Hugh Brendan McMahan. Adaptive federated optimization. In *International Conference on Learning Representations*, 2021. 6

[33] Yasar Abbas Ur Rehman, Yan Gao, Pedro Porto Buarque de Gusmao, Mina Alibeigi, Jiajun Shen, and Nicholas D. Lane. L-dawa: Layer-wise divergence aware weight aggregation in federated self-supervised visual representation learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 16464–16473, 2023. 1, 2, 3, 4, 5, 6, 7, 8

[34] Nicola Rieke, Jonny Hancox, Wenqi Li, Fausto Milletari, Holger R Roth, Shadi Albarqouni, Spyridon Bakas, Mathieu N Galtier, Bennett A Landman, Klaus Maier-Hein, et al. The future of digital health with federated learning. *NPJ digital medicine*, 3(1):1–7, 2020. 1

[35] Xinyi Shang, Yang Lu, Gang Huang, and Hanzi Wang. Federated learning on heterogeneous and long-tailed data via classifier re-training with federated features. 2, 3

[36] Jiangming Shi, Shanshan Zheng, Xiangbo Yin, Yang Lu, Yuan Xie, and Yanyun Qu. Clip-guided federated learning on heterogeneity and long-tailed data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 14955–14963, 2024. 3

[37] Yue Tan, Guodong Long, Lu Liu, Tianyi Zhou, Qinghua Lu, Jing Jiang, and Chengqi Zhang. Fedproto: Federated prototype learning across heterogeneous clients. In *Proceedings of the AAAI conference on artificial intelligence*, pages 8432–8440, 2022. 3

[38] Hongyi Wang, Mikhail Yurochkin, Yuekai Sun, Dimitris Papailiopoulos, and Yasaman Khazaeni. Federated learning with matched averaging. In *International Conference on Learning Representations*, 2020. 7

[39] Jianyu Wang, Zachary Charles, Zheng Xu, Gauri Joshi, H Brendan McMahan, Maruan Al-Shedivat, Galen Andrew, Salman Avestimehr, Katharine Daly, Deepesh Data, et al. A field guide to federated optimization. *arXiv preprint arXiv:2107.06917*, 2021. 1, 3

[40] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pages 23965–23998. PMLR, 2022. 2

[41] Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36, 2024. 2

[42] Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. Adamerging: Adaptive model merging for multi-task learning. In *The Twelfth International Conference on Learning Representations*, 2024. 2, 4

[43] Rui Ye, Mingkai Xu, Jianyu Wang, Chenxin Xu, Siheng Chen, and Yanfeng Wang. Feddisco: Federated learning with discrepancy-aware collaboration. In *Proceedings of the 40th International Conference on Machine Learning*. JMLR.org, 2023. 1, 2, 3, 6, 7

[44] Mikhail Yurochkin, Mayank Agarwal, Soumya Ghosh, Kristjan Greenewald, Nghia Hoang, and Yasaman Khazaeni. Bayesian nonparametric federated learning of neural networks. In *International conference on machine learning*, pages 7252–7261. PMLR, 2019. 7

[45] Sergey Zagoruyko and Nikos Komodakis. Wide Residual Networks. In *British Machine Vision Conference 2016*, York, France, 2016. British Machine Vision Association. 7

[46] Chen Zhang, Yu Xie, Hang Bai, Bin Yu, Weihong Li, and Yuan Gao. A survey on federated learning. *Knowledge-Based Systems*, 216:106775, 2021. 1

[47] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28, 2015. 3